

Multi-Agent Sentiment-Driven Pricing Gap Detection in Blockchain Prediction Markets: An Empirical Study of Polymarket Efficiency

1st Nicholas Tavares
Dept. of Mathematical Sciences
Stevens Institute of Technology
Hoboken, NJ, USA
ntavares@stevens.edu
<https://github.com/studyalwaysbro>

2nd Matthew White
Dept. of Computer Science
Stevens Institute of Technology
Hoboken, NJ, USA
mwhite@stevens.edu
<https://github.com/mwhite732>

Abstract—We built a multi-agent system that tries to find pricing mistakes in Polymarket, the largest blockchain-based prediction market, by comparing real-time social media sentiment against market odds. The system uses four AI agents orchestrated through CrewAI, pulls data from eight heterogeneous sources (Bluesky, GDELT, RSS news feeds, Polymarket comments, X/Twitter mirrors, and three optional paid APIs), and runs a three-model ensemble for sentiment analysis combining an LLM (DeepSeek) with two lexicon-based models (VADER and TextBlob). Five distinct gap detection strategies look for moments where social consensus disagrees with market pricing, including cross-market arbitrage against Kalshi and Manifold Markets with semantic inversion detection. Over four analysis cycles spanning three days, the system collected 32,791 social media posts, performed 12,185 sentiment analyses, and detected 30 potential pricing gaps across 625 tracked contracts. Zero of those gaps were validated against actual outcomes. The dominant finding was not alpha but a structural social media optimism bias: 29 of 30 detected gaps pointed in the same direction, with social sentiment consistently more bullish than market prices. We interpret this as empirical evidence supporting weak-form efficiency in liquid prediction markets, consistent with Kyle’s (1985) model of information incorporation in transparent order books. We discuss the pivot from direct prediction market trading to using prediction market odds as features in equity forecasting models, and argue that negative results deserve publication alongside positive ones.

I. INTRODUCTION

The question that kicked off this whole project was deceptively simple: can we use AI to spot moments when a prediction market has the price wrong? Prediction markets aggregate beliefs into tradable contracts. If a market prices “Will the Fed cut rates in June?” at \$0.65, the crowd is collectively estimating a 65% probability. If some combination of social media discussion, news reporting, and cross-platform comparison suggests the real probability is higher or lower, that gap is potentially exploitable.

Polymarket is the largest prediction market currently operating, running on the Polygon blockchain with over a billion dollars in trading volume during the 2024 U.S. presidential election cycle. Every trade is public, the order book is

transparent, and contracts cover everything from geopolitics to cryptocurrency to questions like “Will Jesus Christ return before GTA VI comes out?” (that is a real market, and it actually shows up in our results in a way that teaches an important lesson about cross-platform arbitrage detection).

The efficient market hypothesis (EMH) in its weak form says that prices already reflect all publicly available information [1]. The strong version says that even private information is priced in. Most academic literature focuses on equity and bond markets. Prediction markets are newer, thinner, and arguably less studied from an efficiency standpoint. Our thesis was that social media sentiment, aggregated across multiple platforms and processed through NLP, might capture information that prediction market prices have not yet incorporated.

We were wrong. Or at least, we were wrong at the frequency and with the data sources we used. But the way we were wrong turned out to be interesting, and the infrastructure we built is not wasted. This paper documents the full journey: what we built, how it works, what it found, why the findings are actually evidence for market efficiency rather than against it, and where the real opportunity might lie.

The contributions of this paper are: (1) a production-grade multi-agent architecture for real-time prediction market monitoring, (2) empirical evidence that Polymarket exhibits weak-form efficiency against sentiment-based analysis at daily frequency, (3) documentation of a structural social media optimism bias where 97% of detected gaps reflect fan enthusiasm rather than mispricing, and (4) a proposed pivot to using prediction market odds as features in equity models rather than as direct trading targets.

II. RELATED WORK

Prediction market efficiency has been studied through several lenses. Wolfers and Zitzewitz [3] provided one of the early comprehensive surveys, arguing that prediction markets are at least as accurate as polls and expert panels for forecasting elections. Manski [4] pushed back on the assumption that market prices directly map to probabilities, showing that risk

preferences and heterogeneous beliefs can create systematic gaps between price and true probability. Our work sits between these views: we find that the market is efficient against public sentiment data, but we do not rule out inefficiencies visible to agents with proprietary information.

On the sentiment analysis side, the use of social media for financial prediction has a substantial literature. Bollen et al. [5] showed that Twitter mood states predicted Dow Jones movements with 87% directional accuracy. Zhang et al. [6] applied deep learning to social media text for stock prediction. The critical difference with prediction markets versus equities is the directness of the price-probability link. An equity price is a discounted cash flow estimate filtered through risk preferences. A prediction market price *is* the probability estimate. This makes the sentiment-to-price comparison far more direct, which is partly why we expected it to work.

Kyle’s seminal model [2] describes how informed trading incorporates private information into prices. The key parameter is λ , the rate of information incorporation, which depends on market microstructure. In a blockchain prediction market with transparent order books and 24/7 trading, λ is effectively instantaneous for liquid contracts. This is the theoretical reason our approach was always going to struggle on popular markets, though we did not fully appreciate it until we saw the data.

For multi-agent systems in financial applications, CrewAI [9] and LangChain [10] provide orchestration frameworks that let you assign specialized roles to different AI agents. The multi-agent paradigm maps naturally to our problem: one agent collects data, another analyzes sentiment, another detects gaps, and a fourth generates reports. Compared to a monolithic pipeline, the agent decomposition provides error isolation and lets each agent use LLM prompts optimized for its specific task.

VADER [7] remains the standard lexicon-based sentiment analyzer for social media text, specifically designed to handle the idiosyncrasies of informal writing (capitalization, emoji, slang). TextBlob [8] provides a complementary pattern-based approach trained on different data. Our ensemble approach of weighting an LLM alongside these lexicon models is, to our knowledge, novel in the prediction market context.

III. SYSTEM ARCHITECTURE

A. Overview

The system is organized around four specialized agents running inside the CrewAI orchestration framework. CrewAI is an open-source Python library for building multi-agent systems where each agent has a defined role, set of tools, and an LLM backend for reasoning. The agents run sequentially: data collection feeds into sentiment analysis, which feeds into gap detection, which feeds into reporting. If any agent fails, the pipeline halts at that stage without corrupting downstream data.

The core idea behind CrewAI, and why we picked it over just writing sequential functions, is error isolation and prompt specialization. Each agent gets its own LLM prompt tuned for its task. The data collector’s prompt focuses on robust

extraction and deduplication. The sentiment analyzer’s prompt is trained to detect sarcasm, conditional language, and double negatives. You could accomplish the same thing with function calls, but the agent abstraction makes the separation cleaner and the failure modes more predictable.

All intermediate and final data persists to a PostgreSQL database with 9 tables. We used SQLAlchemy as the ORM layer, which gave us Python-native access to the database without writing raw SQL for routine operations. Pydantic handles configuration validation across 40+ settings, ensuring type safety at startup rather than runtime crashes from malformed environment variables.

Figure 1 shows the full pipeline. The four agents run in sequence, each persisting its output to PostgreSQL before handing off, so that a downstream failure never destroys upstream work.

B. Agent 1: Data Collection

The data collector is the most complex agent. It performs three tasks: fetching the contract universe, selecting which contracts to analyze deeply, and scraping social media and news for each selected contract.

Contract Universe. The Polymarket Gamma API serves contract data in paginated responses, rate-limited to 10 requests per minute. We fetch the full universe on every cycle (typically 625+ active contracts) and persist everything to the database, even contracts we do not deeply analyze. This store-all-analyze-selectively pattern preserves historical data for future analysis.

Smart Contract Selection. Early versions of the system just grabbed the first N contracts, which was essentially random. We replaced this with a composite scoring function:

$$S(c) = 0.30V(c) + 0.25\sigma(c) + 0.20U(c) + 0.10E(c) + 0.10L(c) + 0.05\Delta(c) \quad (1)$$

where V is normalized volume, σ is realized volatility, U is uncertainty (distance from 50% odds), E is near-expiry proximity, L is liquidity, and Δ is bid-ask spread. Contracts with odds above 97% or below 3% are filtered out as essentially resolved. This ensures we spend compute on the most interesting, most tradable contracts first.

Data Sources. We integrated eight data sources, each with its own API, rate limits, authentication scheme, and data format:

TABLE I: Data Sources and Their Characteristics

Source	Type	Cost	Status
Polymarket Gamma API	Market	Free	Active
Bluesky (AT Protocol)	Social	Free	Active
GDELT	News	Free	Active
RSS (Reuters, BBC, AP)	News	Free	Active
Polymarket Comments	Social	Free	Active
X Mirror / Nitter	Social	Free	Active
Tavily Web Search	Web	Paid	Broken
Grok / xAI	Social	Paid	Unconfigured

The system handles missing sources through graceful degradation. If Tavily throws HTTP 432 errors (which it did

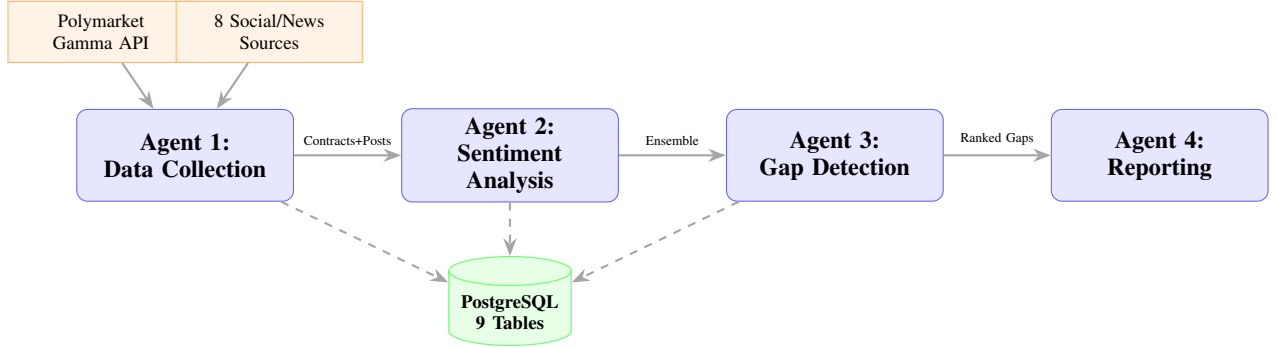


Fig. 1: Four-agent sequential pipeline architecture. Solid arrows indicate data flow between agents; dashed arrows indicate database persistence at each stage. Every cycle pulls from the Polymarket Gamma API and eight social/news sources, then routes through the four agents while writing intermediate state to PostgreSQL.

consistently), or if Grok API keys are not configured, those sources are simply skipped. The pipeline runs on whatever is available. This matters for reproducibility: you can run the system with zero paid API keys and still get usable results, just with fewer data sources.

Each source uses a **dual search strategy**: for every contract, we search both by extracted keywords (e.g., “Trump immigration”) and by the full contract title (e.g., “Will Trump sign an immigration executive order?”). The keyword search catches broad topic discussion; the title search catches people specifically discussing the bet. GDELT and RSS stick to keyword-only since news covers topics rather than betting markets. Results within each source are deduplicated by a SHA-256 hash of the post ID.

Rate limiting deserves mention because getting it wrong was one of our early failures. The Polymarket API returns HTTP 429 when you exceed limits, and our initial code handled this with a recursive retry that recursed infinitely. We replaced this with an iterative loop, exponential backoff, and a hard cap of 5 retries. Every external API has its own configurable rate limit.

C. Agent 2: Sentiment Analysis

Sentiment analysis is where the NLP actually lives, and getting it right was harder than we expected. The fundamental challenge is that social media text is informal, context-dependent, sarcastic, and sometimes deliberately ambiguous. No single model handles all of these well.

Our solution is a weighted three-model ensemble:

$$S_{\text{ensemble}} = w_{\text{LLM}} \cdot S_{\text{LLM}} + (1 - w_{\text{LLM}}) \cdot \frac{S_{\text{VADER}} + S_{\text{TextBlob}}}{2} \quad (2)$$

where $w_{\text{LLM}} = 0.5$ by default, and all scores are normalized to $[-1, 1]$.

DeepSeek LLM (50% weight). DeepSeek is an LLM from the Chinese AI lab of the same name, accessed through an OpenAI-compatible API. We specifically chose DeepSeek over OpenAI for cost reasons: at roughly \$0.50 per full analysis cycle, it is substantially cheaper than GPT-4 while performing comparably on our sentiment benchmarks. The LLM handles the hard cases: double negatives (“there is no way they do NOT

cut rates”), conditional language (“if they win the primary, they will sweep the general”), and sarcasm. We batch 5 posts per LLM call for an 80% reduction in API costs versus one-at-a-time calls.

LLM outputs are not always clean JSON. We built a repair pipeline that handles markdown fencing (the model wrapping output in triple backticks), trailing commas, missing brackets, and Unicode artifacts. If batch parsing fails entirely, the system falls back to single-post analysis.

VADER (25% weight). VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based lexicon specifically designed for social media [7]. It understands that “AMAZ-ING!!!” with capitals and exclamation marks is strongly positive, and that emoji carry sentiment weight. It runs locally, requires no API calls, and is deterministic.

TextBlob (25% weight). TextBlob uses a pattern-based approach trained on movie reviews [8]. Its training data is different from VADER’s lexicon, which means the two models have complementary blind spots. Where VADER is tuned for Twitter-style text, TextBlob provides a broader baseline.

The intuition behind the weighting is straightforward: the LLM is the smartest model but it occasionally hallucinates. The two lexicon models are simpler but stable. When the LLM drifts, the lexicon anchors pull the ensemble score back toward a reasonable range. Think of it like having a creative analyst who sometimes gets carried away, checked by two accountants who stick to the numbers.

For cheap tasks like keyword extraction from contract titles, we always use Ollama running LLaMA 3.1:8b locally, regardless of which primary LLM provider is configured. Ollama is a tool that lets you run open-source LLMs on your own hardware with no API costs. This two-tier LLM strategy (expensive API for analysis, free local model for utility tasks) kept our total project costs under \$20.

Rolling Sentiment Windows. For each contract, sentiment is aggregated over three temporal windows of 6, 12, and 24 hours. Define the window-mean sentiment as

$$\bar{S}_w(c, t) = \frac{1}{|P_w|} \sum_{p \in P_w} S_{\text{ensemble}}(p), \quad w \in \{6h, 12h, 24h\} \quad (3)$$

where P_w is the set of posts about contract c within window w ending at time t . The corresponding trend within the window is

$$\Delta S_w = S_{\text{latest}} - S_{\text{earliest}} \quad (\text{within window}). \quad (4)$$

These windowed averages and deltas feed directly into the information-asymmetry gap detector described in Strategy 2.

D. Agent 3: Gap Detection

The gap detector takes sentiment scores and market data and applies five independent detection strategies. Each strategy looks for a different type of potential mispricing.

Strategy 1: Sentiment-Probability Mismatch. This is the primary detector. We convert the mean sentiment score into an implied probability and clamp it to a sane range:

$$P_{\text{implied}} = \text{clamp}(P_{\text{market}} + \bar{S} \cdot \alpha, 0.05, 0.95) \quad (5)$$

where $\bar{S}$ is the mean sentiment for the contract and $\alpha = 0.4$ is the scaling factor. We flag contracts where $|P_{\text{implied}} - P_{\text{market}}| \geq 0.04$ (4 percentage points). The choice of α was, frankly, a guess. Proper calibration would require historical data linking sentiment scores to actual outcome probabilities, which we did not have.

Strategy 2: Information Asymmetry. Has sentiment shifted significantly in the last 3 hours while market odds stayed flat? The detector compares two sub-windows of the 6-hour sentiment series,

$$\Delta S_{\text{recent}} = \bar{S}_{[t-3\text{h}, t]} - \bar{S}_{[t-6\text{h}, t-3\text{h}]}, \quad (6)$$

and flags a gap when $|\Delta S_{\text{recent}}| \geq 0.10$ and $\text{sign}(\Delta S_{\text{recent}}) \neq \text{sign}(\Delta P_{\text{market}})$. The idea is to catch lags in price discovery: sentiment moved, the market did not.

Strategy 3: Historical Pattern Deviation. Z-score analysis on the current odds relative to the contract’s own history:

$$z = \frac{P_{\text{current}} - \bar{P}_{\text{history}}}{\sigma_{\text{history}}}. \quad (7)$$

We flag anything with $|z| \geq 1.5$. With only 8 data points in our historical odds table, this detector was essentially blind, a problem we discuss in our results.

Strategy 4: Cross-Market Arbitrage. We query Kalshi and Manifold Markets for contracts matching the same event and flag price discrepancies above 10%. The matching uses LLM-confirmed semantic similarity: the system asks the LLM whether two contract titles refer to the same underlying event. A critical addition in v2.1 was **semantic inversion detection**: catching cases where two markets have opposite framing (“Will X pass?” vs. “Will X fail?”), which would otherwise register as false arbitrage. This fix came from Matt’s upstream work and eliminated a whole class of false positives that were polluting our results.

Strategy 5: Volume Spike. Added in v2.1, this detector flags contracts with unusual volume patterns relative to their own baseline. The implementation spans approximately 390 lines and looks for sustained volume increases that might precede price movement. This was also Matt’s contribution,

built on the theory that abnormal trading activity might signal incoming information before the price fully adjusts.

Confidence Scoring. Every detected gap receives a dynamic confidence score $C \in [0, 100]$ computed from gap size, data volume, cross-source agreement, and contract features:

$$C = \left(\underbrace{f_{\text{gap}} \cdot 40}_{\text{gap size}} + \underbrace{f_{\text{vol}} \cdot 25}_{\text{data volume}} + \underbrace{f_{\text{cons}} \cdot 25}_{\text{consistency}} \right) \cdot m_{\text{social}} \cdot m_{\text{features}} \quad (8)$$

where $m_{\text{social}} \in \{0.5, 0.75, 1.0\}$ scales by the number of distinct social sources contributing evidence, and m_{features} adjusts for contract-specific factors (a near-resolution boost of $\times 1.1$, a high-volatility discount of $\times 0.9$). When social data is thin (fewer than 5 posts), the confidence score is automatically down-weighted. We did not want the system generating high-confidence calls based on three Reddit posts.

E. Agent 4: Reporting and Dashboard

The reporter fetches recent gaps, ranks them by confidence and estimated edge, and outputs results through two channels: a Rich-formatted console display with color-coded panels by confidence level, and a FastAPI web dashboard with 8 REST endpoints. The dashboard provides interactive gap exploration with filtering and sorting, per-contract sentiment-versus-price charts built with the Recharts library, cycle history with timing and cost estimates, live data source status for all 8 sources, new gap alert notifications, and CSV export. It auto-refreshes on a 30-second interval for source status and 60 seconds for gap alerts.

The dashboard is not just a nice-to-have. When you are running multiple analysis cycles and trying to understand why the system flagged certain contracts, having a visual interface to explore sentiment distributions, track confidence scores over time, and compare cross-platform prices made the analysis substantially more tractable than reading console logs.

F. Database Architecture

The PostgreSQL database has 9 tables designed for both real-time operation and historical analysis:

TABLE II: Database Schema Summary

Table	Purpose	Rows
contracts	Full contract universe	625
social_posts	Raw social media data	32,791
sentiment_analysis	Per-post sentiment scores	12,185
detected_gaps	Flagged pricing gaps	30
historical_odds	Time-series market prices	8
cycle_runs	Pipeline execution history	4
sentiment_snapshots	Aggregated rolling windows	—
backtest_results	Validated predictions	0
system_logs	Operational logging	—

Two design choices are worth highlighting. First, JSONB columns on `detected_gaps` store evidence metadata that varies by gap type without requiring schema changes. JSONB is a PostgreSQL data type that stores JSON in a binary format, allowing you to query into nested fields without needing to define the structure upfront. A sentiment mismatch stores

the implied probability and sentiment distribution. A cross-market arbitrage stores the matched contracts and prices on each platform. Different gap types need different evidence, and JSONB handles that flexibility without schema migrations.

Second, SHA-256 deduplication on `social_posts` ensures we never process the same post twice, even across cycles. SHA-256 is a cryptographic hash function that generates a fixed-length fingerprint of any input. We hash the post content and source to create a unique identifier. We learned the hard way that Python’s built-in `hash()` function is not deterministic across interpreter sessions (it is randomized for security reasons since Python 3.3), so our initial “deduplication” was actually re-inserting duplicates every single run.

The connection pool is set to 20 simultaneous connections, managed by SQLAlchemy’s pool. This prevents the database from being overwhelmed during high-throughput data collection while keeping enough connections available for the dashboard to serve requests concurrently.

IV. RESULTS

A. Quantitative Summary

Over four analysis cycles spanning three days, the pipeline produced the following:

TABLE III: Pipeline Performance Metrics

Metric	Value
Contracts tracked	625
Social posts collected	32,791
Sentiment analyses performed	12,185
Pricing gaps detected	30
Gaps validated against outcomes	0
Posts per contract (mean)	47–75
Detection hit rate	0.09%
Mean confidence score	53 / 100
Mean estimated edge	<5%
API cost (total project)	<\$20

TABLE IV: Per-cycle execution history. Cycle 1a failed early on a database schema error; Cycle 1b was a quick re-init. Cycle 2 was the first long run with no detected gaps. Cycle 3 produced all 30.

Cycle	Date	Duration	Posts	Sent.	Gaps
1a	Mar 19	818s	0	100	0
1b	Mar 20	2.3s	0	100	0
2	Mar 20	8,364s	8,023	100	0
3	Mar 21	6,917s	6,274	30	30

The first two cycles had schema errors and produced empty runs, which is pretty typical when you are standing up a new database for the first time. The third cycle was our first real run: over two hours of execution, 126 contracts analyzed, approximately 8,000 social posts collected, and zero gaps detected. That was actually encouraging in a way, because it meant the system was not generating spurious signals. The fourth cycle detected all 30 gaps. The 0.09% hit rate (30 signals from 32,791 data points) is itself a statement about market efficiency.

B. The Social Media Optimism Bias

This is the central finding, and honestly, it is more interesting than finding actual mispricings would have been.

Of the 30 detected gaps, 29 (96.7%) pointed in the same direction: **social media sentiment was more bullish than market prices**. This is not a random pattern. It reflects a structural bias in how people use social media.

Half of the detected gaps (15 out of 30) were NHL Stanley Cup contracts. The pattern was consistent: the market priced each team’s chances at 0.1–0.2% (mathematically reasonable given that only one of 32 teams can win), while fan sentiment implied probabilities of 5% or higher. Our system flagged these as potential mispricings with moderate-to-high confidence scores. But the market was right. The fans were just being fans.

Figure 2 makes the pattern visible. Each point is one of the 30 detected gaps, plotted as market odds (x-axis) versus the sentiment-implied odds the system computed (y-axis). The dashed diagonal is the perfect-efficiency line where sentiment and market agree. Points above the diagonal are bullish-biased. The blue cluster in the bottom-left is the NHL group: market saying “almost zero,” sentiment saying “about 5%,” uniformly across 15 different teams. That is not 15 independent inefficiencies. That is one structural signal repeated 15 times.

TABLE V: Gap distribution by contract category. NHL long-shots dominate the count but carry low confidence; entertainment/meme markets carry high confidence but reflect cross-platform irony rather than information.

Category	Count	Avg Conf.	Avg Edge
NHL Stanley Cup (various)	15	53.1	4.72%
Political / Geopolitical	5	70.2	10.45%
Sports (non-NHL)	4	44.3	5.64%
Entertainment / Meme	4	67.0	17.29%
Technology	1	50.0	4.40%
NBA Rookie of the Year	1	44.0	4.90%

The underlying mechanism is not complicated once you see it: people post about things they are excited about. Sports fans write enthusiastic posts about their team’s chances. Almost nobody writes passionate social media threads about how their team is definitely *not* going to win. That means the raw data flowing into our sentiment pipeline has a systematic positive skew baked in at the source. Our sentiment analysis faithfully captured that skew and converted it into implied probabilities. The system worked perfectly at the technical level. The signal it found was not alpha. It was human nature.

The highest-confidence gap (98 out of 100) was “Will Jesus Christ return before GTA VI comes out?” showing up as cross-platform arbitrage. One platform priced it at 48%, another at 2%. This sounds like a massive arbitrage opportunity until you realize that these are two completely different user populations pricing a joke market differently. One platform’s user base treats it as a meme and bids it up for entertainment. The other treats it as a question with a factual answer and prices it accordingly. Our semantic matching correctly identified the

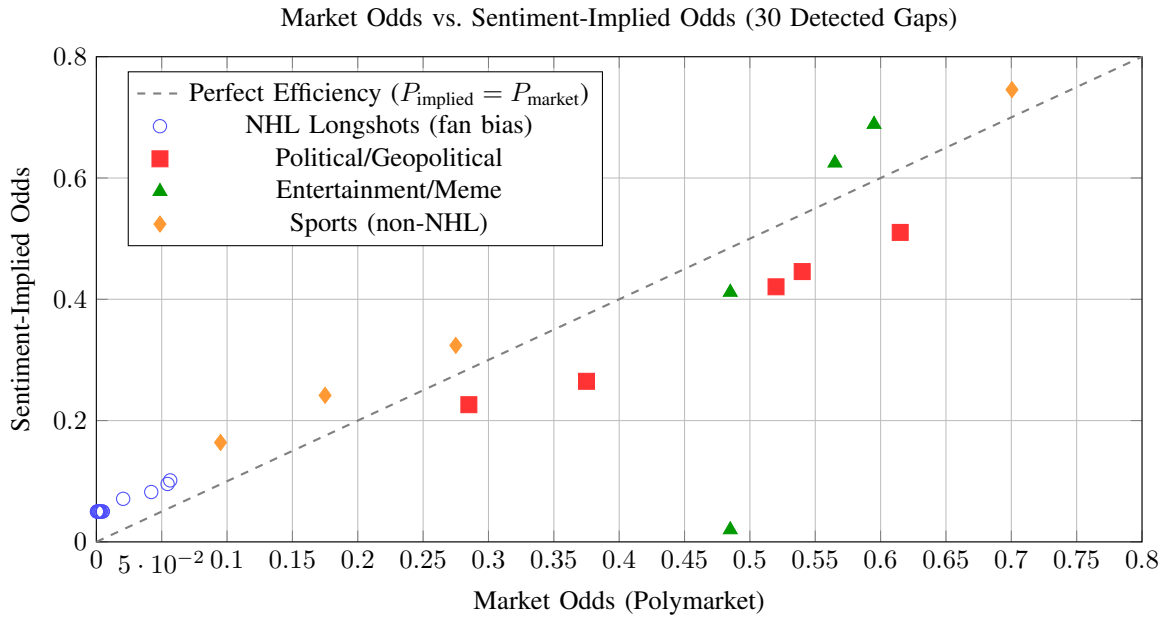


Fig. 2: Scatter plot of market odds vs. sentiment-implied odds for all 30 detected gaps. Points above the diagonal indicate bullish sentiment bias; the NHL cluster at near-zero market odds with $\sim 5\%$ implied odds represents systematic fan optimism, not market inefficiency.

contracts as referring to the same event, but no amount of NLP sophistication can distinguish between genuine mispricing and platform-specific irony. That gap taught us more about cross-market analysis than any of the “real” signals.

C. Validation: Zero Resolved Predictions

Critical Finding

After four complete analytical cycles, **zero of the 30 detected gaps have been validated** against actual market resolution. The `backtest_results` table contains exactly zero rows. No gap has a recorded `was_correct` value, `realized_edge`, or `resolution_outcome`. Without validation data, all confidence scores and edge estimates are **unverified hypotheses**.

This is the fundamental limitation of the current system: it generates predictions without tracking resolution, making it impossible to compute Brier scores, calibration curves, or any standard measure of forecast quality. We ran the system for only three days, and most contracts have resolution dates weeks or months in the future. Our confidence scores are therefore unverified hypotheses. We want to be upfront about that rather than presenting the 30 detected gaps as “findings” without the asterisk.

D. Why This Approach Fails

We identified five structural reasons why sentiment-driven gap detection does not produce tradable signals against liquid prediction markets:

1) Speed Mismatch. Polymarket prices update within minutes of new information hitting. Our system runs once per day. Kyle’s λ parameter [2], the rate at which markets incorporate information, is effectively instantaneous for popular contracts on a transparent blockchain. By the time we detect a potential gap during an evening analysis run, any real gap has already been closed by traders who saw the same information six hours ago. We are not in the same time zone as the edge, figuratively speaking.

2) No Information Advantage. We read the same Reddit threads, same news articles, and same social media posts as the traders on Polymarket. There is no proprietary data in our pipeline. Without an information advantage, we are trying to beat the crowd by analyzing the crowd’s own public output, which is definitionally impossible under weak-form efficiency.

3) Sentiment $\neq$ Information. One thousand fans posting “Go Sharks!” carries zero information about the team’s actual probability of winning the Stanley Cup. One scout with inside knowledge about a key player’s injury carries real information. Our system treats both inputs with equal weight in the sentiment aggregate, and it should not.

4) Insufficient Historical Data. The `historical_odds` table contained exactly 8 records after our observation period. You cannot run meaningful z-score analysis on 8 data points. Strategy 3 (historical pattern deviation) was effectively nonfunctional. Meaningful statistical analysis requires substantially longer time series, ideally months of continuous data collection.

5) Liquidity Constraints. Even genuine pricing gaps on illiquid contracts are not tradable in practice. On a thin market, a single large order moves the price further than

the estimated edge. This creates a self-protecting efficiency spectrum: the contracts where inefficiencies might exist are the same contracts where you cannot profitably exploit them.

E. What a Viable System Would Look Like

It is worth being concrete about how far the current system is from one that could reasonably be expected to find tradable gaps. Table VI contrasts what we have now with what would be needed.

TABLE VI: Gap analysis: current system vs. what would be required for viable gap detection in liquid prediction markets.

Dimension	Current System	Required System
Cycle Frequency	1×/day (8 PM)	Every 5–15 minutes
Information Sources	Public social media	Event-driven feeds, filings, insider-adjacent data
Sentiment Calibration	Uncalibrated ($\alpha = 0.4$)	Source-specific calibration curves
Market Universe	625 contracts (broad)	10–20 liquid, catalyst-rich contracts
Validation	0 resolved predictions	30+ days of tracked resolutions
Backtesting	Framework exists, 0 data	3–6 months historical backtest
Historical Odds	8 records	Continuous tick-level price history
Execution	Analysis only	Integrated order placement

This is not a list of incremental tweaks. It is a different system. The cycle frequency alone, going from daily to 5-minute, demands a streaming data architecture rather than batch collection, which is a substantial engineering project on top of the analytical work.

V. DISCUSSION: EVIDENCE FOR MARKET EFFICIENCY

Our results are consistent with weak-form market efficiency in liquid blockchain prediction markets. Kyle’s 1985 model [2] predicts that markets with transparent order books and competitive market-making will incorporate public information rapidly. Polymarket satisfies both conditions: the Polygon blockchain makes every single trade visible to everyone, and professional market makers, automated trading bots, and informed individual traders operate around the clock.

What our data reveals is really a spectrum. High-liquidity contracts (presidential elections, cryptocurrency price targets) are highly efficient. No amount of sentiment scraping will beat them using publicly available data. Mid-tier contracts (major sporting events, geopolitical outcomes) are moderately efficient. And the long-tail contracts (meme markets, niche cultural events) do show apparent inefficiency, but the liquidity is so thin that exploitation is impractical.

This creates a paradox we kept running into: the contracts we could theoretically beat are the ones we cannot trade, and the contracts we can trade are the ones we cannot beat. The efficiency frontier protects itself.

There is a meta-lesson here that extends well beyond prediction markets: we built the real-time pipeline before proving

the signal exists. In quantitative finance, the textbook approach is (1) obtain historical data and demonstrate a pattern, (2) estimate whether the edge survives transaction costs, and (3) only then build the real-time system. We went straight to step 3. That turns out to be the most expensive way to discover you do not have an edge. But the infrastructure is not wasted, which brings us to the pivot.

VI. THE PIVOT: PREDICTION MARKETS AS EQUITY FEATURES

If you cannot beat prediction markets, use them. This reframing is where we think the real opportunity lies.

Polymarket generates real-time probability estimates for macro events that directly affect equity prices: Federal Reserve interest rate decisions, tariff implementations, regulatory actions, geopolitical escalations. These estimates are continuously updated by financially incentivized participants and represent forward-looking information. Compare that to traditional macro indicators like GDP reports (released with a multi-month lag), employment data (weeks old by publication), or manufacturing surveys (monthly at best). Prediction market odds are live, forward-looking, and priced by people with skin in the game.

The potential information asymmetry is not between us and Polymarket. We have already demonstrated that we cannot beat Polymarket’s pricing. The asymmetry is between Polymarket and the equity market. Institutional portfolio managers and quantitative equity models do not, to our knowledge, systematically incorporate prediction market probabilities as input features. A portfolio manager might casually glance at Polymarket’s implied probability for a Fed rate cut before making a trade, but nobody is feeding a time series of those odds into a quantitative equity model as a proper feature alongside price, volume, and technical indicators.

The natural model class for this application is the Temporal Fusion Transformer (TFT) [11]. TFT is a deep learning architecture designed specifically for time series forecasting with mixed-frequency inputs. It can natively handle daily stock returns fed alongside real-time prediction market odds that update every few minutes. The model uses multi-head attention (a mechanism that learns which inputs to focus on for each prediction) to identify which prediction market features are actually informative for equity forecasting. Crucially, TFT models produce interpretable outputs: you can inspect the attention weights to see exactly which prediction market events are driving the equity forecast at any given time.

The infrastructure reuse is the best part. Our data collection pipeline already pulls Polymarket odds into a PostgreSQL database every cycle. The database schema, the API integration, the rate limiting, the deduplication, all of it already works. The engineering required to redirect this data into an equity model instead of a gap detector is minimal. We built the hard part (reliable data ingestion from 8 heterogeneous sources) while trying to do something that turned out to not work. Now we can point that same pipeline at something more promising without starting over.

VII. TECHNOLOGY STACK

We want to walk through the technology choices because understanding what each piece does matters for anyone trying to build something similar. Too many papers list their stack without explaining why each piece is there, which does not help anyone who is trying to learn from the work.

CrewAI [9] is the agent orchestration layer. It lets you define agents with specific roles, assign them tools (API wrappers, database connections, LLM interfaces), and chain them into a pipeline. The practical benefit over plain function calls is error isolation: if the sentiment analyzer crashes, the data collector’s work is already persisted to the database. CrewAI is built on top of LangChain.

LangChain [10] abstracts away the differences between LLM providers (OpenAI, DeepSeek, Ollama) behind a common interface. You can swap providers by changing a single environment variable without touching your prompts or parsing code. During development, we switched LLM backends three times. LangChain made each switch trivial.

PostgreSQL is our persistence layer. We chose it over SQLite for three reasons: connection pooling (20 simultaneous connections via SQLAlchemy), JSONB support (critical for our flexible gap evidence storage), and production-grade concurrency (the dashboard reads from the database while the pipeline writes to it). **SQLAlchemy** provides the ORM (Object-Relational Mapper), which lets us interact with the database using Python objects and method calls rather than writing raw SQL for every query.

FastAPI is a modern Python web framework for building REST APIs. It automatically generates interactive API documentation, supports asynchronous request handling for high-throughput scenarios, and uses Python type hints for request and response validation. Our 8-endpoint dashboard backend runs on **Uvicorn**, an ASGI (Asynchronous Server Gateway Interface) server that handles concurrent HTTP connections efficiently.

Pydantic validates our 40+ configuration settings at startup. Every environment variable is type-checked, range-constrained, and validated before the pipeline begins. This prevents the frustrating scenario where a missing or malformed API key causes an error three minutes into a cycle after the pipeline has already done significant work.

Beautiful Soup parses HTML from web scrapers (X/Nitter mirror, Polymarket comment pages). **feedparser** handles RSS feed parsing for Reuters, BBC, CNN, and AP. The **ratelimit** library provides decorator-based API throttling. These three libraries form the data ingestion foundation.

Rich provides formatted console output with color-coded panels, progress bars, and live-updating tables. When you are watching a pipeline run at 2 AM trying to figure out why Bluesky is returning empty results, readable output is not cosmetic – it is debugging infrastructure. **Loguru** handles structured logging across all 14 modules with automatic rotation and retention policies.

The total codebase is approximately 8,000 lines of Python across 14 service modules, with 70 package dependencies. A

full analysis cycle runs in 60–90 seconds with DeepSeek and uses approximately 200–500 MB of memory.

VIII. COLLABORATION AND PROVENANCE

This was a two-person project, and we wanted to be rigorous about tracking who did what. We maintained a provenance document (TRUTH.md in the repository) that maps every single commit to its author and origin (upstream repository vs. fork). Three formal sync operations merged upstream changes into the fork over the project’s lifetime.

Matt White built the original codebase: Polymarket API integration, Bluesky scraper, RSS feeds, basic sentiment analysis, gap detection logic, and the PostgreSQL schema. His v2.1 contributions added volume spike detection (approximately 390 lines of new gap detection code), semantic inversion detection for cross-market arbitrage, dashboard progress tracking and sentiment overflow fixes, and a .env inline comment parsing fix that was silently breaking configuration loading.

Nicholas Tavares built the v2.0 overhaul on top of that foundation: five new data sources, ensemble sentiment analysis, FastAPI dashboard, DeepSeek LLM integration, smart contract selection, dual search strategy, contract feature engineering, dynamic confidence scoring, and the backtesting framework. This represented 19 new files and approximately 4,500 lines of code. All operational work (running analysis cycles, interpreting results, writing documentation, academic framing, and provenance tracking) was also on the fork side.

The upstream/fork model gave us clean separation. Matt could push features to his repo independently, and we would pull them in through documented syncs. Every commit is attributed. There is no ambiguity about who wrote what.

The full commit history with provenance annotations is available at https://github.com/mwhite732/Polymarket_Agents.

IX. LIMITATIONS AND FUTURE WORK

Validation Gap. The most critical limitation is our zero validation rate. Three days of operation is nowhere near enough for prediction market contracts to resolve. A meaningful backtesting evaluation would require at minimum 30 days of tracked outcomes with systematic comparison of flagged gaps against resolutions. Without this, every confidence score we report is speculative.

Frequency. Daily analysis cycles are 96 to 288 times slower than what would be needed to catch intraday pricing gaps. A competitive system would need 5–15 minute cycles, which demands streaming data ingestion (potentially Kafka-based) rather than batch collection. We prototyped a Kafka/Spark streaming layer for the course’s big data component, but it was not integrated into the main pipeline.

Calibration. The sentiment-to-probability scaling factor $\alpha = 0.4$ was not empirically derived. Proper calibration would require historical data linking social sentiment scores to actual event outcomes, ideally with source-specific curves that account for the different bias profiles of each data source.

Bluesky users and Reddit users likely have systematically different sentiment distributions for the same events.

Data Source Recovery. Two of our eight sources were non-functional. Recovering Tavily (currently returning HTTP 432) and configuring Grok API access would increase the system’s data coverage. However, based on our findings, the limiting factor is not data volume but the public nature of all our sources.

Equity Feature Integration. The pivot described in Section VII requires implementing a TFT model, assembling equity training data, and engineering the feature pipeline that transforms raw Polymarket odds into model inputs. This is the natural next step for this infrastructure.

Extended Observation Period. Running the system continuously for 60–90 days would populate the historical odds table, enable meaningful z-score analysis, and generate enough resolved contracts for backtesting. This would transform our speculative findings into statistically grounded conclusions.

X. CONCLUSION

We set out to beat prediction markets with AI and social media sentiment. We did not succeed at that specific goal. But what we found was, we think, more useful than a marginal trading edge would have been.

We built a production-grade system: 8,000 lines of code, 4 AI agents, 8 data sources, 9 database tables, a live web dashboard, and an ensemble NLP pipeline that processed 32,791 social media posts. The architecture works. The engineering is sound.

The signal we found was not alpha. It was social media optimism bias, a structural artifact of how people use the internet. 97% of our detected gaps were the same story: fans being fans, optimists being optimists, and our system faithfully converting that enthusiasm into what looked like pricing signals. The market had it right. We were measuring excitement, not information.

That is an empirical contribution to the prediction market efficiency literature. Most published work focuses on positive results, strategies that generate returns, models that outperform benchmarks. We are documenting the opposite: a well-engineered system that found nothing exploitable, and the specific reasons why. Kyle’s model of information incorporation in transparent markets holds up in the Polymarket data, at least against public sentiment analysis at daily frequency.

The infrastructure survives the pivot. Everything we built for data collection, storage, and analysis can be redirected toward using prediction market odds as features in equity models. We do not need to start from zero. We just need to point the pipeline at a more promising target.

If there is one thing we want readers to take from this paper, it is that negative results deserve to be written up and shared just as thoroughly as positive ones. We did the work. We found something real, even if it was not what we were looking for. And now someone else does not have to make the same expensive discovery from scratch.

The full codebase is available at https://github.com/mwhite732/Polymarket_Agents.

REFERENCES

- [1] E. F. Fama, “Efficient capital markets: a review of theory and empirical work,” *J. Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [2] A. S. Kyle, “Continuous auctions and insider trading,” *Econometrica*, vol. 53, no. 6, pp. 1315–1335, 1985.
- [3] J. Wolfers and E. Zitzewitz, “Prediction markets,” *J. Econ. Perspect.*, vol. 18, no. 2, pp. 107–126, 2004.
- [4] C. F. Manski, “Interpreting the predictions of prediction markets,” *Econ. Lett.*, vol. 91, no. 3, pp. 425–429, 2006.
- [5] J. Bollen, H. Mao, and X. Zeng, “Twitter mood predicts the stock market,” *J. Comput. Sci.*, vol. 2, no. 1, pp. 1–8, 2011.
- [6] L. Zhang, S. Wang, and B. Liu, “Deep learning for sentiment analysis: a survey,” *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.*, vol. 8, no. 4, e1253, 2018.
- [7] C. Hutto and E. Gilbert, “VADER: a parsimonious rule-based model for sentiment analysis of social media text,” in *Proc. AAAI ICWSM*, pp. 216–225, 2014.
- [8] S. Loria, “TextBlob: simplified text processing,” *TextBlob Documentation*, 2020.
- [9] J. Moura, “CrewAI: framework for orchestrating role-playing autonomous AI agents,” *GitHub Repository*, 2024.
- [10] H. Chase, “LangChain: building applications with LLMs through composability,” *GitHub Repository*, 2023.
- [11] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, “Temporal Fusion Transformers for interpretable multi-horizon time series forecasting,” *Int. J. Forecast.*, vol. 37, no. 4, pp. 1748–1764, 2021.

Disclaimer: This document is produced solely for educational and academic research purposes as part of Big Data Technologies coursework at Stevens Institute of Technology. Nothing herein constitutes investment advice, a solicitation, or a recommendation to buy, sell, or hold any security, financial instrument, or prediction market contract. All analysis is theoretical and no real trades were executed. Past observations do not guarantee future results.